Data Transformation Assignment

Objective
Prepare the given dataset for Machine Learning by applying:

- Encoding
- Normalization
- Standardization (Scaling)
- Preprocessing Pipeline

Dataset: Employee_productivity_Dataset

Instructions

- Use Pandas and Scikit-learn
- Perform steps in order
- Display output after each question
- Write short explanations where asked

Assignment Questions

Q1. Handle Missing Values (Basic Cleaning)
Fill missing values using the following:
Age → Median
Salary → Mean
Hours_Worked_Per_Week → Median
Performance_Score → Mean
Display the dataset after handling missing values.

Answer:

```
import pandas as pd

# Filling missing values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] =
df['Hours_Worked_Per_Week'].fillna(df['Age'].median())
df['Performance_Score'] =
df['Performance_Score'].fillna(df['Performance_Score'].mean())
```

```
print(df.head())
```

Q2. Label Encoding
Convert these columns into numeric values using Label Encoding:
Gender
Department
Show the updated columns.

Answer:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])
df['Department'] = le.fit_transform(df['Department'])

print(df[['Gender', 'Department']].head())
```

Q3. One-Hot Encoding
Apply One-Hot Encoding on:
Work_Mode
Location
Display the dataset and check how many new columns are created.

Answer:

```
# Applying One-Hot Encoding
df = pd.get_dummies(df, columns=['Work_Mode', 'Location'])

print(df.head())
print(f"Total columns after One-Hot Encoding: {df.shape[1]}")
```

Q4. Normalization (Min-Max Scaling)
Normalize the following columns:
Salary Hours_Worked_Per_Week
Use MinMaxScaler and display results.

Answer:

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
df[['Salary', 'Hours_Worked_Per_Week']] = scaler.fit_transform(df[['Salary',
'Hours_Worked_Per_Week']])

print(df[['Salary', 'Hours_Worked_Per_Week']].head())
```

Q5. Standardization (Scaling)
Apply StandardScaler on:
Age
Projects_Completed
Display the transformed values.

Answer:

```
from sklearn.preprocessing import StandardScaler

std_scaler = StandardScaler()
df[['Age', 'Projects_Completed']] = std_scaler.fit_transform(df[['Age',
'Projects_Completed']])

print(df[['Age', 'Projects_Completed']].head())
```

Q6. Compare Scaling Methods
Apply both:
MinMaxScaler
StandardScaler
On the Salary column.
Show both results side by side.

Answer:

```
salary_data = df[['Salary']] # Assuming raw salary for comparison

m_scaler = MinMaxScaler()
s_scaler = StandardScaler()

comparison_df = pd.DataFrame({
    'MinMax_Salary': m_scaler.fit_transform(salary_data).flatten(),
    'Standard_Salary': s_scaler.fit_transform(salary_data).flatten()
})
```

```
print(comparison_df.head())
```

Q7. Build Preprocessing Pipeline
Create a pipeline that: Applies encoding to categorical columns Applies scaling to numerical columns Use: ColumnTransformer Pipeline

Answer:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Define features
num_features = ['Age', 'Salary', 'Hours_Worked_Per_Week', 'Projects_Completed']
cat_features = ['Work_Mode', 'Location']

# Define transformers
num_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine into ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', num_transformer, num_features),
        ('cat', cat_transformer, cat_features)
    ])

print("Pipeline Created Successfully.")
```

Q8. Apply Pipeline
Apply the pipeline on the dataset. Display: Transformed dataset Shape of final dataset

Answer:

```
# Assuming X is your feature set
X_transformed = preprocessor.fit_transform(df)

print("Transformed Dataset (First 5 rows):")
print(X_transformed[:5])
print(f"Final Dataset Shape: {X_transformed.shape}")
```

Q9. Conceptual Question
Why is scaling important in Python?

Answer:

Scaling is vital because most Machine Learning algorithms (like KNN, SVM, and Gradient Descent-based models) calculate the distance between data points. If one feature has a range of 0-1 (like Age) and another has 0-100,000 (like Salary), the algorithm will be biased toward the larger numbers, treating the smaller feature as irrelevant "noise." Scaling ensures all features contribute equally to the model's learning process.

Q10. Conceptual Question
Why do we convert categorical data into numerical form?

Answer:

Mathematical equations and Machine Learning models cannot perform arithmetic operations on "Strings" (like "Remote" or "New York"). They require numerical input to calculate weights, gradients, and distances. Converting text to numbers (encoding) bridges the gap between human-readable data and machine-executable math.